

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Нейроинформатика
Лабораторная работа №3

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Русак Алена Викторовна

Содержание	
Задание.....	3
Ход работы.....	4
Ход эксперимента.....	4
Исходный код.....	21
Вывод.....	22

Задание

1. Исследовать применение различных архитектур нейронных сетей для классификации новостей.
2. Попробовать улучшить предсказание моделей за счет использования не только текста, но и заголовка новости. При этом рекомендуется увеличить максимальную длину текста.
3. Проверить, как на качество предсказания влияет предобработка текста (удаление стоп-слов, пунктуации, лемматизация и др.)
4. Применить методы регуляризации для устранения переобучения моделей. Для наглядного отслеживания результатов и переобучения рекомендуется использовать графики изменения точности и функции потерь.
5. Подготовить отчет с результатами проведенных экспериментов.

Ход работы

Для работы использовался датасет "AG's News Topic Classification Dataset", содержащий новости, их заголовки и принадлежность классу. В качестве архитектур моделей использовались одномерная сверточная нейронная сеть (CNN), рекуррентная нейросеть LSTM и рекуррентная нейронная сеть GRU.

Ход эксперимента

Сначала была произведена базовая предобработка датасета, для которой был создан свой токенизатор, который приводит текст в вектор признаков.

Листинг 1 - Базовая предобработка датасета.

```
# Максимальное количество уникальных слов слов
num_words = 10000
# Максимальная длина новости
max_news_len = 30
# Количество классов новостей
nb_classes = 4
train = pd.read_csv('train.csv',
                    header=None,
                    names=['class', 'title', 'text'])

news = train['text']
y_train = utils.to_categorical(train['class'] - 1, nb_classes)
vectorize_layer =
tf.keras.layers.TextVectorization(max_tokens=num_words,
    output_mode='int',
    output_sequence_length=max_news_len)
vectorize_layer.adapt(news)
tokenizer = Tokenizer(num_words=num_words)
tokenizer.fit_on_texts(news)
sequences = tokenizer.texts_to_sequences(news)
x_train = pad_sequences(sequences, maxlen=max_news_len)
```

Далее были обучены и протестированы 3 модели. На листингах 2-7 представлена реализация моделей и результаты их тестирования.

Листинг 2 - Реализация одномерной свёрточной нейронной сети.

```
model_cnn = keras.Sequential()
model_cnn.add(keras.layers.Embedding(num_words, 32,
input_length=max_news_len))
model_cnn.add(keras.layers.Conv1D(250, 5, padding='valid',
activation='relu'))
model_cnn.add(keras.layers.GlobalMaxPooling1D())
model_cnn.add(keras.layers.Dense(128, activation='relu'))
model_cnn.add(keras.layers.Dense(4, activation='softmax'))
model_cnn.compile(optimizer='adam',
loss='categorical_crossentropy',
metrics=['accuracy'])
# model_cnn_save_path = 'best_model_cnn.h5'
checkpoint_filepath_cnn = '/tmp/ckpt/checkpoint.model_cnn.keras'
checkpoint_callback_cnn = ModelCheckpoint(checkpoint_filepath_cnn,
monitor='val_accuracy',
save_best_only=True,
verbose=1)

history_cnn = model_cnn.fit(x_train,
y_train,
epochs=15,
batch_size=128,
validation_split=0.1,
callbacks=[checkpoint_callback_cnn])
model_cnn.load_weights(checkpoint_filepath_cnn)
model_cnn.evaluate(x_test, y_test, verbose=1)
```

Листинг 3 - Результат тестирования CNN.

```
238/238 ————— 1s 4ms/step - accuracy:
0.8892 - loss: 0.3340
[0.3118465840816498, 0.8931578993797302]
```

Листинг 4 - Реализация модели LSTM.

```
model_lstm = keras.Sequential()
model_lstm.add(keras.layers.Embedding(num_words, 32,
input_length=max_news_len))
model_lstm.add(keras.layers.LSTM(16))
model_lstm.add(keras.layers.Dense(4, activation='softmax'))
model_lstm.compile(optimizer='adam',
loss='categorical_crossentropy',
metrics=['accuracy'])
```

```
# model_lstm_save_path = 'best_model_lstm.h5'
checkpoint_filepath_lstm = '/tmp/ckpt/checkpoint.model_lstm.keras'
checkpoint_callback_lstm = ModelCheckpoint(checkpoint_filepath_lstm,
                                          monitor='val_accuracy',
                                          save_best_only=True,
                                          verbose=1)

history_lstm = model_lstm.fit(x_train,
                              y_train,
                              epochs=15,
                              batch_size=128,
                              validation_split=0.1,
                              callbacks=[checkpoint_callback_lstm])

model_lstm.load_weights(checkpoint_filepath_lstm)
model_lstm.evaluate(x_test, y_test, verbose=1)
```

Листинг 5 - Результат тестирования LSTM.

```
238/238 _____ 1s 4ms/step - accuracy:
0.8828 - loss: 0.3559
[0.3337469696998596, 0.8915789723396301]
```

Листинг 6 - Реализация модели GRU.

```
model_gru = keras.Sequential()
model_gru.add(keras.layers.Embedding(num_words, 32,
input_length=max_news_len))
model_gru.add(keras.layers.GRU(16))
model_gru.add(keras.layers.Dense(4, activation='softmax'))
model_gru.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
# model_gru_save_path = 'best_model_gru.h5'
checkpoint_filepath_gru = '/tmp/ckpt/checkpoint.model_gru.keras'
checkpoint_callback_gru = ModelCheckpoint(checkpoint_filepath_gru,
                                          monitor='val_accuracy',
                                          save_best_only=True,
                                          verbose=1)

history_gru = model_gru.fit(x_train,
                            y_train,
                            epochs=15,
                            batch_size=128,
                            validation_split=0.1,
                            callbacks=[checkpoint_callback_gru])

model_gru.load_weights(checkpoint_filepath_gru)
```

```
model_gru.evaluate(x_test, y_test, verbose=1)
```

Листинг 7 - Результат тестирования GRU.

```
238/238 _____ 1s 4ms/step - accuracy:
0.8864 - loss: 0.3485
[0.33086857199668884, 0.8903947472572327]
```

Далее на рисунках 1-3 представлены графики изменения точности во время обучения и валидации моделей.

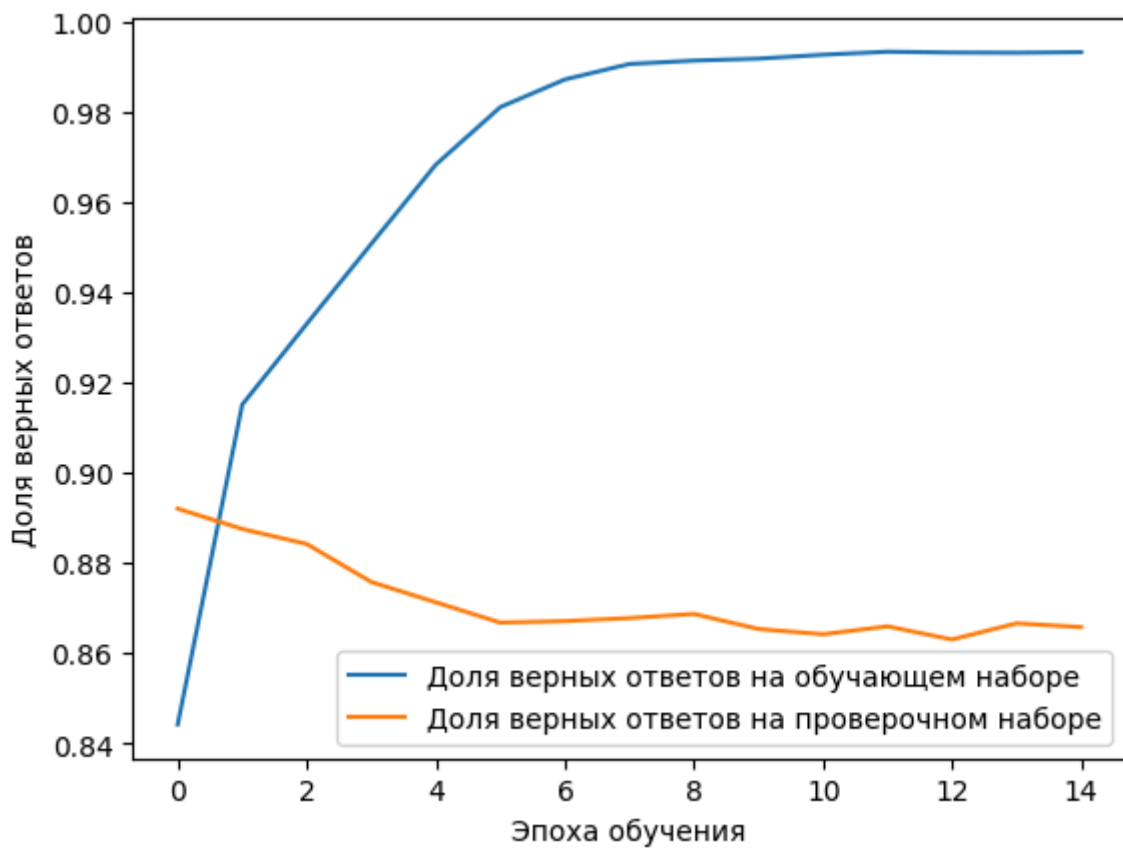


Рисунок 1 - Изменение accuracy модели CNN.

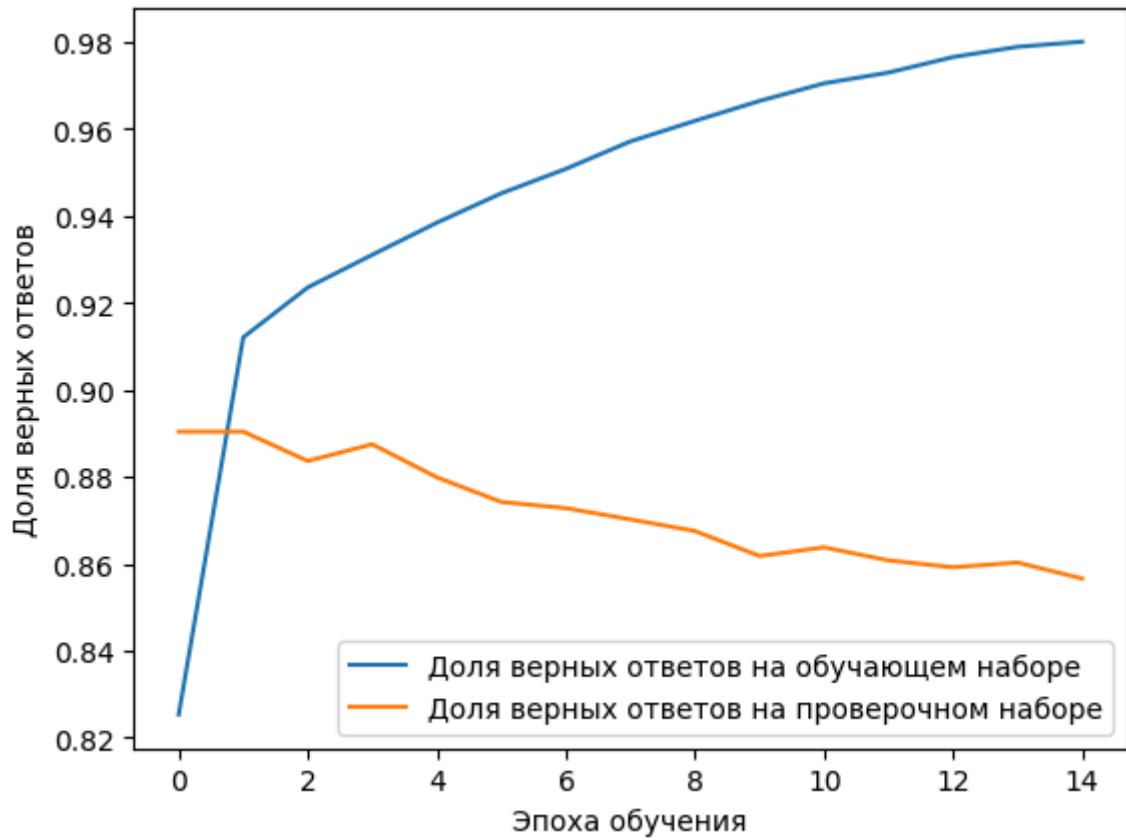


Рисунок 2 - Изменение ассигасу модели LSTM.

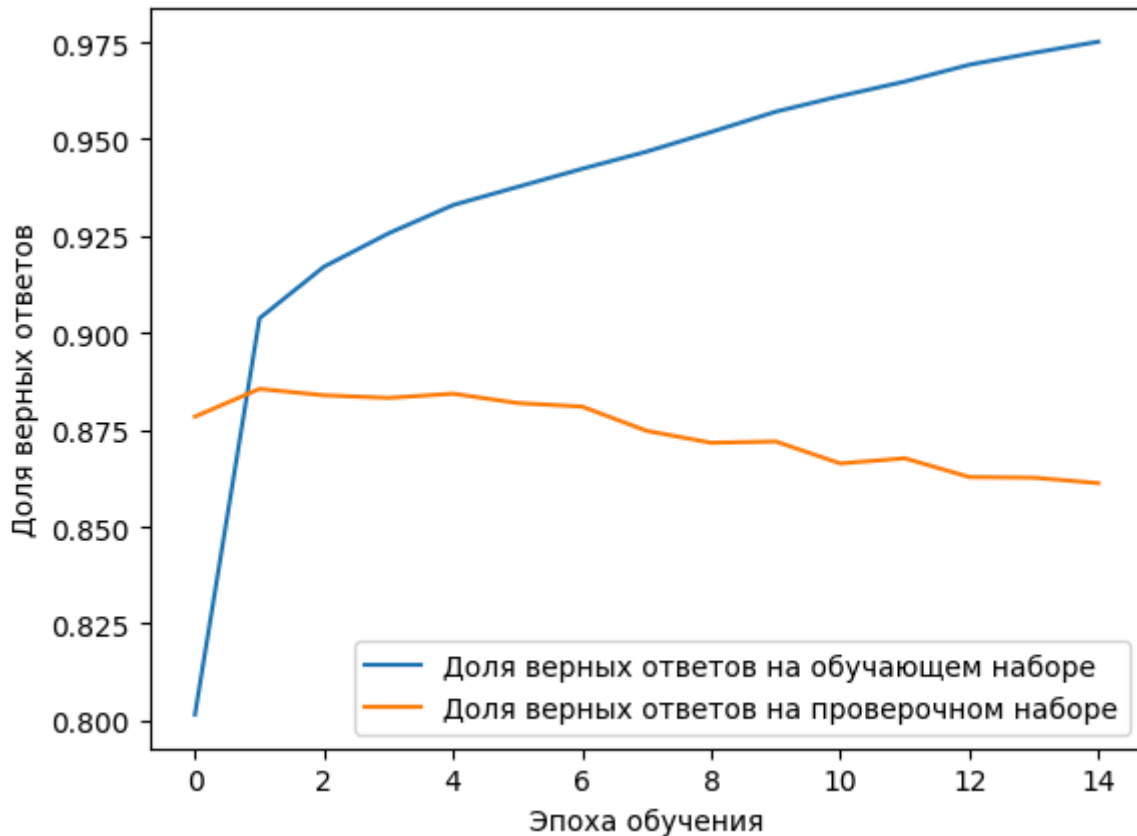


Рисунок 3 - Изменение accuracy модели GRU.

Далее в рамках эксперимента добавим к тексту новостей их заголовки, соответственно увеличим максимальную длину текста. Далее на листинге 8 представлена новая предобработка текста.

Листинг 8 - Предобработка текста с добавлением заголовка новости.

```
longer_news = train['title'] + ' ' + train['text']
new_max_news_len = 60
vectorize_layer_1 =
tf.keras.layers.TextVectorization(max_tokens=num_words,
    output_mode='int',
    output_sequence_length=new_max_news_len)

vectorize_layer_1.adapt(longer_news)
tokenizer_1 = Tokenizer(num_words=num_words)
tokenizer_1.fit_on_texts(longer_news)
sequences_1 = tokenizer_1.texts_to_sequences(longer_news)
x_train_1 = pad_sequences(sequences_1, maxlen=new_max_news_len)
```

На новых данных модели были переобучены без изменения архитектур. На листингах 9-11 представлены результаты их тестирования, а на рисунках 4-6 изменение точности в течение обучения и валидации.

Листинг 9 - Результат тестирования CNN.

```
238/238 ----- 1s 4ms/step - accuracy:
0.8972 - loss: 0.3099
[0.2956238090991974, 0.8980262875556946]
```

Листинг 10 - Результат тестирования LSTM.

```
238/238 ----- 1s 5ms/step - accuracy:
0.8793 - loss: 0.3565
[0.33439379930496216, 0.8877631425857544]
```

Листинг 11 - Результат тестирования GRU.

```
238/238 ----- 1s 5ms/step - accuracy:
0.8885 - loss: 0.3383
[0.3179928660392761, 0.893815815448761]
```

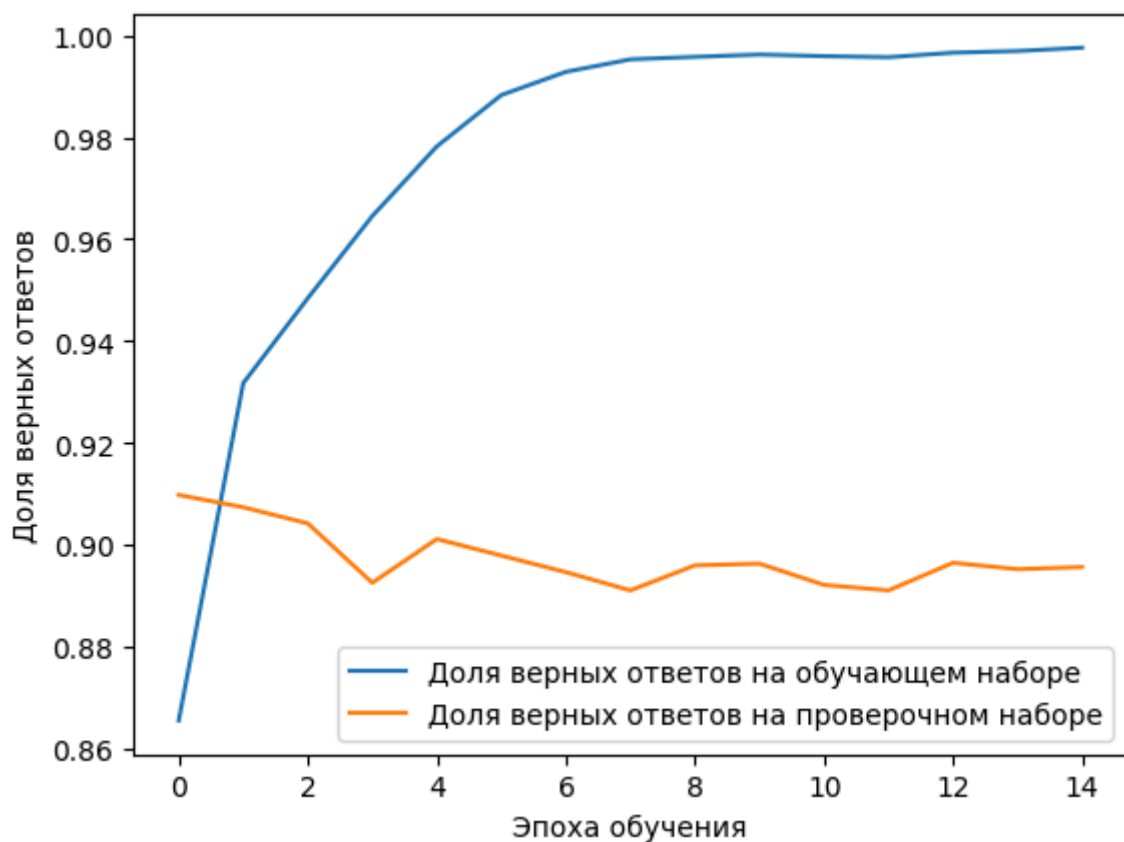


Рисунок 4 - Изменение accuracy модели CNN.

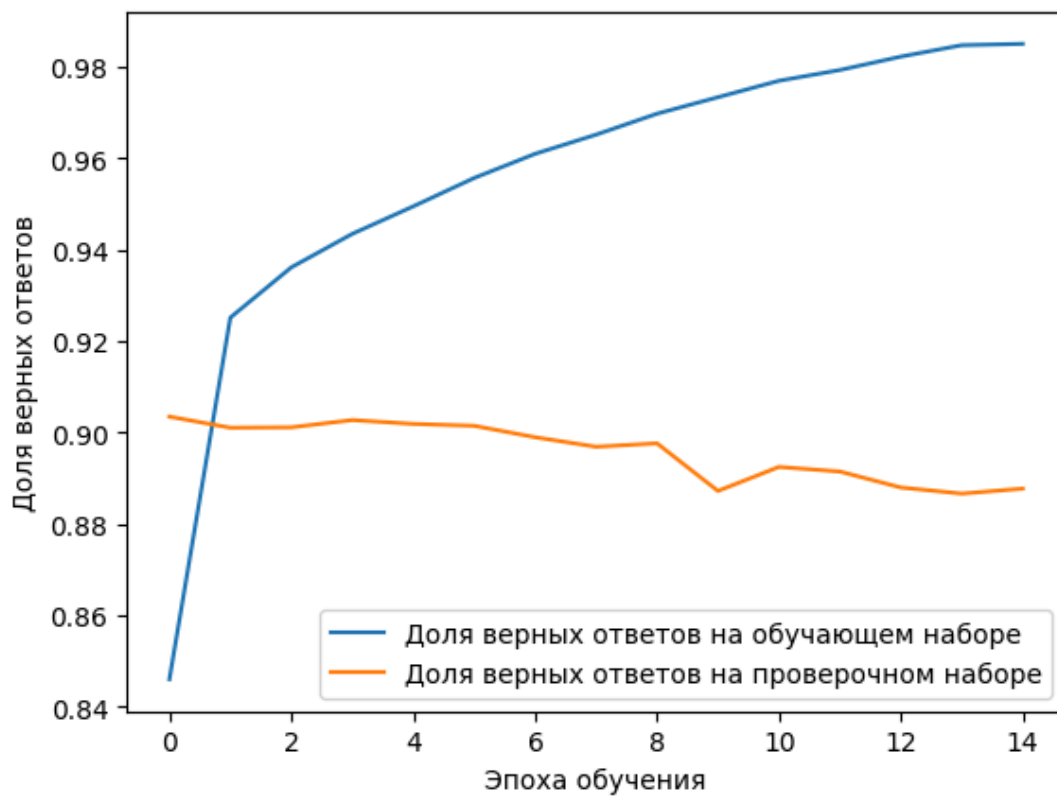


Рисунок 5 - Изменение ассигасу модели LSTM.

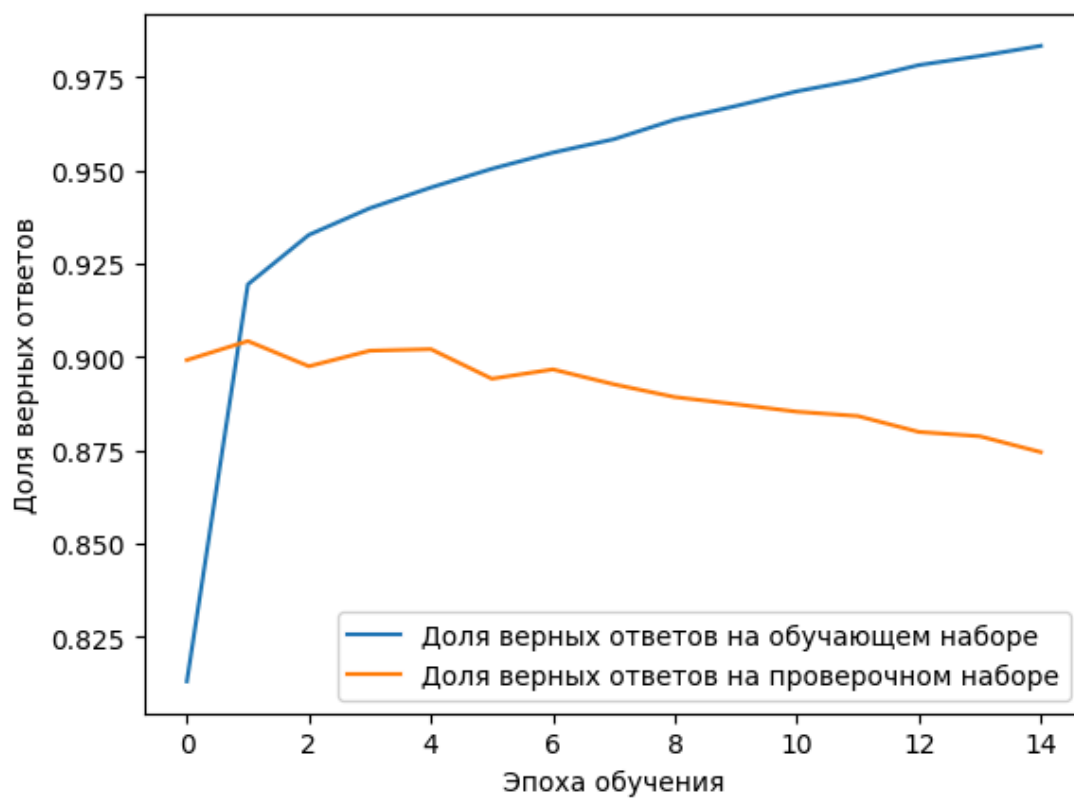


Рисунок 6 - Изменение ассигасу модели GRU.

Следующим шагом в экспериментах стала очистка текста от стоп-слов и пунктуации. Стоп-слова - это слова, которые часто встречаются практически в любом тексте и не несут полезной информации о конкретном документе. Для модели это просто шум. По аналогичной причине убираем и пунктуацию. Далее на листинге 12 представлена обработка текста для данной задачи.

Листинг 12 - Очистка текста в датасете от стоп-слов и пунктуации.

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
# импортируем стоп-слова из библиотеки nltk
from nltk.corpus import stopwords

# посмотрим на стоп-слова для английского языка
print(stopwords.words('english'))

from string import punctuation
punctuation

noise = stopwords.words('english') + list(punctuation)
# Оставляем только слова не из noise

news_clean = []

for text in longer_news:
    tokens = text.lower().split()
    tokens = [word for word in tokens if word not in noise]
    clean_text = " ".join(tokens)
    news_clean.append(clean_text)

vectorize_layer_2 =
tf.keras.layers.TextVectorization(max_tokens=num_words,
    output_mode='int',
    output_sequence_length=new_max_news_len)

vectorize_layer_2.adapt(news_clean)
vectorize_layer_2.get_vocabulary()
tokenizer_2 = Tokenizer(num_words=num_words)
tokenizer_2.fit_on_texts(news_clean)
sequences_2 = tokenizer_2.texts_to_sequences(news_clean)
```

```
x_train_2 = pad_sequences(sequences_2, maxlen=new_max_news_len)
```

Модели снова были переобучены уже на “чистом” тексте, включая заголовки. На листингах 13-15 представлены результаты их тестирования, а на рисунках 7-9 изменение точности в течение обучения и валидации.

Листинг 13 - Результат тестирования CNN.

```
238/238 _____ 1s 4ms/step - accuracy:
0.7983 - loss: 0.5168
[0.5103085041046143, 0.7939473390579224]
```

Листинг 14 - Результат тестирования LSTM.

```
238/238 _____ 1s 5ms/step - accuracy:
0.8258 - loss: 0.5138
[0.5099401473999023, 0.8219736814498901]
```

Листинг 15 - Результат тестирования GRU.

```
238/238 _____ 1s 5ms/step - accuracy:
0.7418 - loss: 0.7685
[0.7495254874229431, 0.7410526275634766]
```

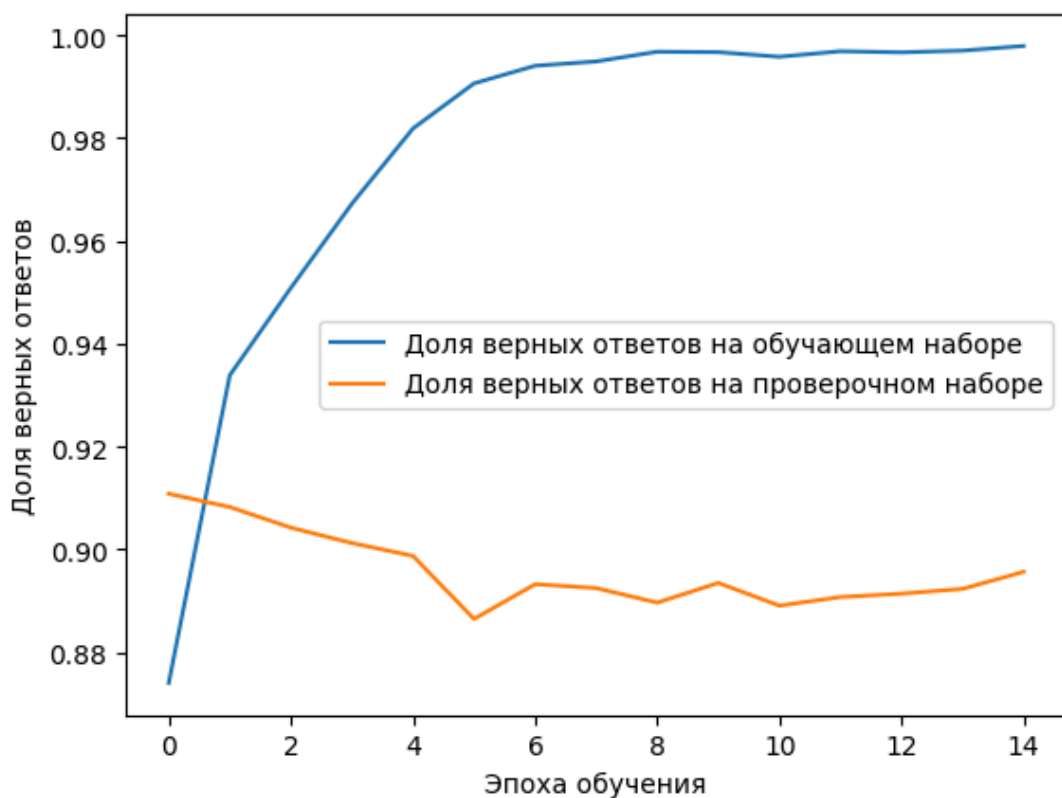


Рисунок 7 - Изменение ассурасу модели CNN.

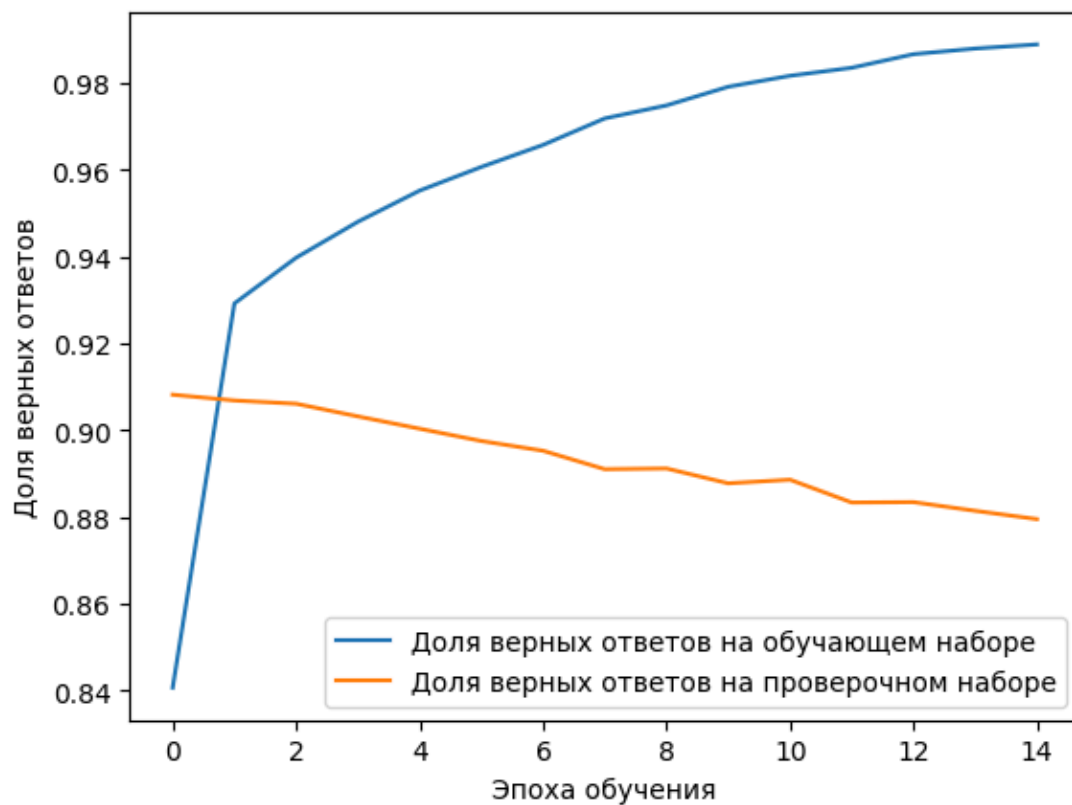


Рисунок 8 - Изменение ассигасу модели LSTM.

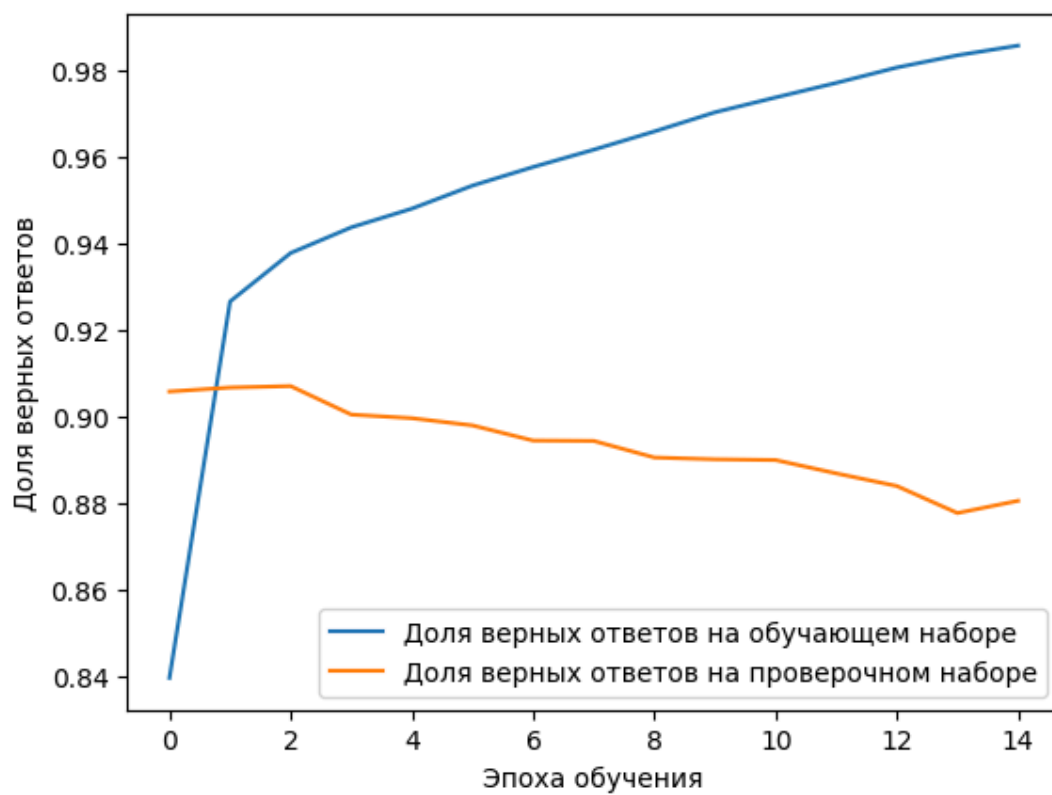


Рисунок 9 - Изменение ассурасу модели GRU.

Далее в рамках эксперимента проведем лемматизацию, сведение разных форм одного слова к начальной форме – лемме, исходного текста с заголовками. На листинге 14 представлена новая предобработка текста.

Листинг 14 - Предобработка текста лемматизацией.

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lem_news = list(lemmatizer.lemmatize(word) for word in longer_news)
vectorize_layer_3 =
tf.keras.layers.TextVectorization(max_tokens=num_words,
    output_mode='int',
    output_sequence_length=new_max_news_len)

vectorize_layer_3.adapt(lem_news)
tokenizer_3 = Tokenizer(num_words=num_words)
tokenizer_3.fit_on_texts(lem_news)
sequences_3 = tokenizer_3.texts_to_sequences(lem_news)
x_train_3 = pad_sequences(sequences_3, maxlen=new_max_news_len)
```

Модели также были переобучены с учетом лемматизации. На листингах 16-18 представлены результаты их тестирования, а на рисунках 10-12 изменение точности в течение обучения и валидации.

Листинг 16 - Результат тестирования CNN.

```
238/238 ————— 1s 3ms/step - accuracy:
0.8891 - loss: 0.3240
[0.29971903562545776, 0.8965789675712585]
```

Листинг 17 - Результат тестирования LSTM.

```
238/238 ————— 1s 5ms/step - accuracy:
0.8944 - loss: 0.3268
[0.3088304102420807, 0.8971052765846252]
```

Листинг 18 - Результат тестирования GRU.

```
238/238 ————— 1s 5ms/step - accuracy:
0.8934 - loss: 0.3290
[0.31151434779167175, 0.8971052765846252]
```

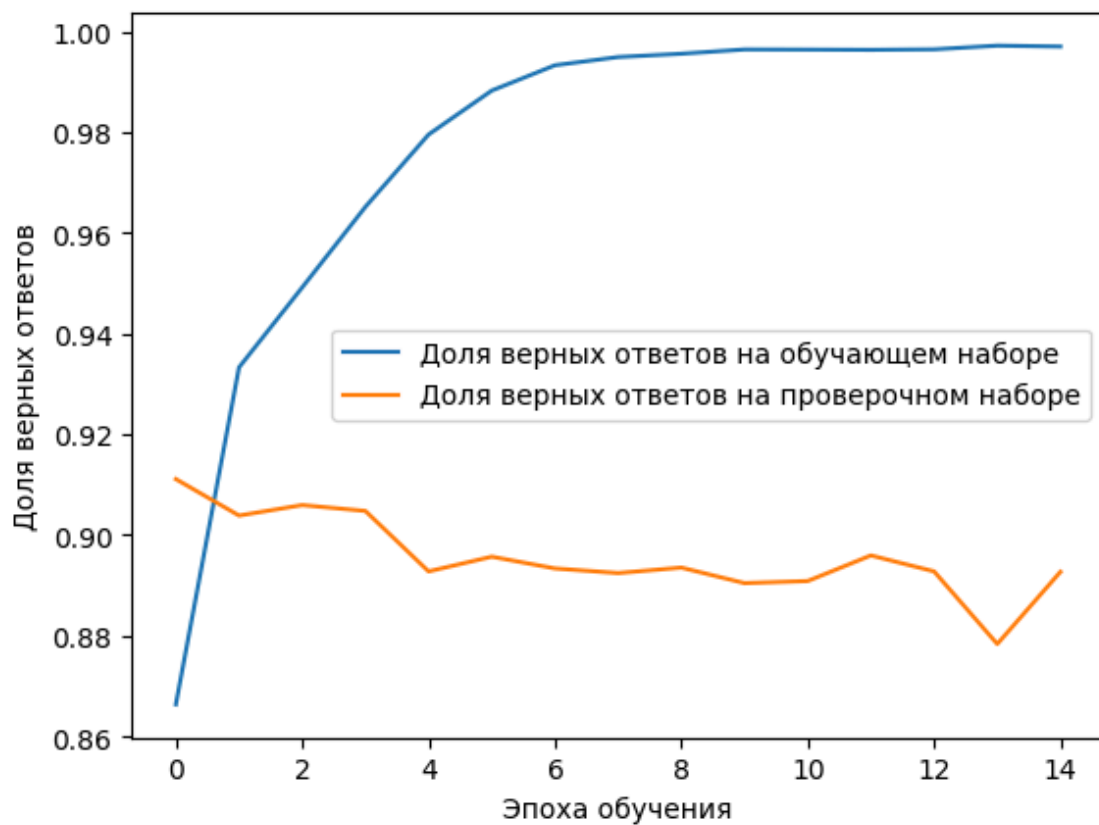


Рисунок 10 - Изменение ассигуры модели CNN.

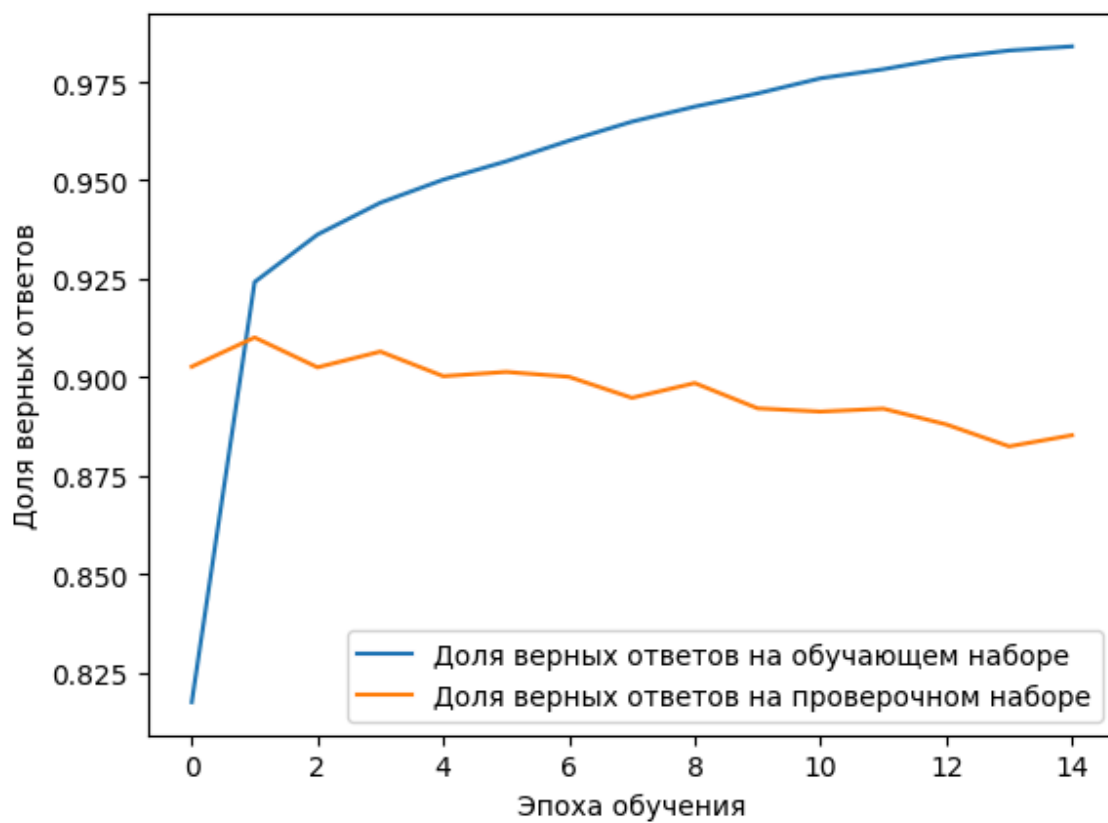


Рисунок 11 - Изменение ассигуры модели LSTM.

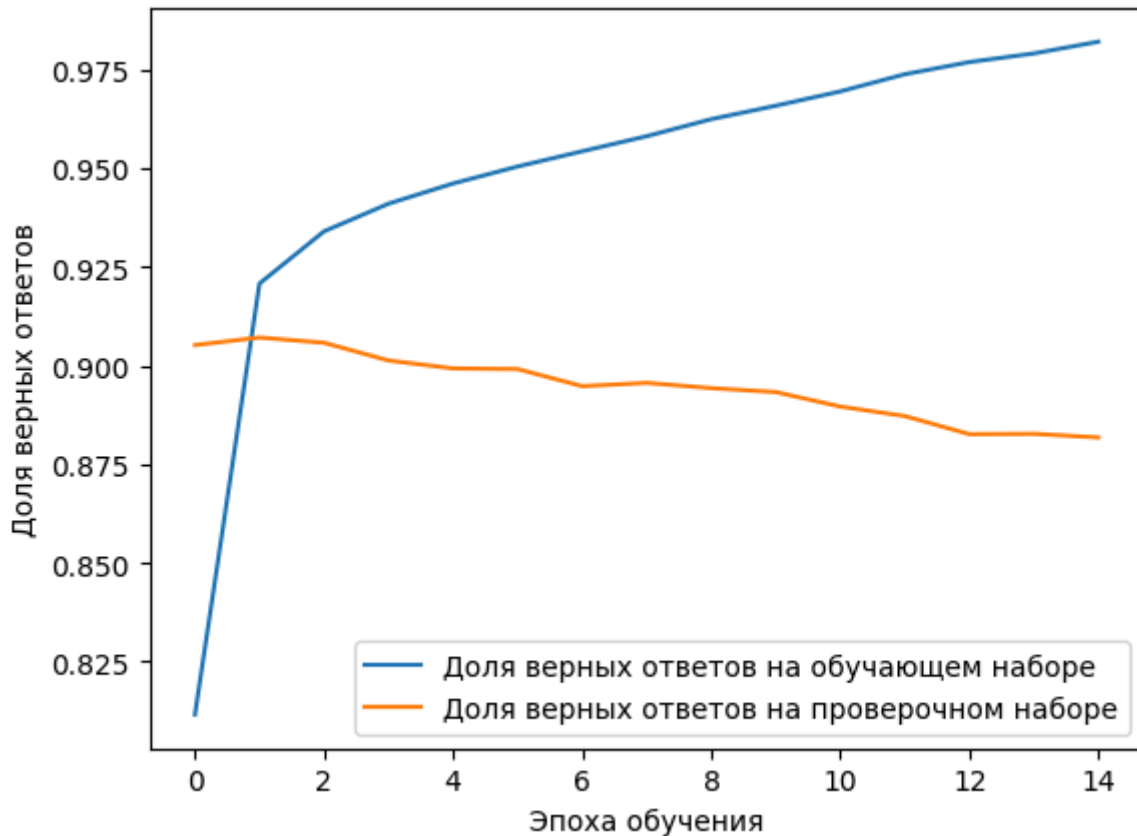


Рисунок 12 - Изменение ассюрасу модели GRU.

Далее я решила применить регуляризацию к некоторым моделям. Исходя из графиков было выдвинуто предположение о том, что к переобучению наиболее склонны свёрточные нейронные сети. Немного изменим архитектуру модели и обучим её на каждом из предобработанных датасетов. На листинге 19 представлена измененная архитектура с регуляризацией.

Листинг 19 - Реализация CNN с регуляризацией.

```
model_cnn_1 = keras.Sequential()
model_cnn_1.add(keras.layers.Embedding(num_words, 32,
input_length=new_max_news_len))
model_cnn_1.add(keras.layers.Conv1D(250, 5, padding='valid',
activation='relu'))
model_cnn_1.add(keras.layers.Dropout(0.3))
model_cnn_1.add(keras.layers.GlobalMaxPooling1D())
model_cnn_1.add(keras.layers.Dense(128, activation='relu'))
model_cnn_1.add(keras.layers.Dropout(0.5))
model_cnn_1.add(keras.layers.Dense(4, activation='softmax'))
```

```

model_cnn_1.compile(optimizer='adam',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
history_cnn_1 = model_cnn.fit(x_train_1,
                              y_train,
                              epochs=5,
                              batch_size=128,
                              validation_split=0.1,
                              callbacks=[checkpoint_callback_cnn_1])
model_cnn_1.evaluate(x_test_1, y_test, verbose=1)

```

Далее на листингах 20-23 представлены результаты тестирования модели, обученной на разных датасетах. Также на рисунках 13 - 16 представлены графики изменения точности модели во время обучения и валидации.

Листинг 20 - Результат тестирования на датасете с базовой предобработкой.

```

238/238 ————— 1s 4ms/step - accuracy:
0.8978 - loss: 0.3773
[0.3598794639110565, 0.8857894539833069]

```

Листинг 21 - Результат тестирования на датасете с добавлением заголовков.

```

238/238 ————— 2s 4ms/step - accuracy:
0.8863 - loss: 1.3863
[1.386339783668518, 0.24828948080539703]

```

Листинг 22 - Результат тестирования на датасете с удалением стоп-слов.

```

238/238 ————— 2s 4ms/step - accuracy:
0.8715 - loss: 1.3861
[1.3868780136108398, 0.25]

```

Листинг 23 - Результат тестирования на датасете с лемматизацией.

```

238/238 ————— 2s 4ms/step - accuracy:
0.8893 - loss: 1.3876
[1.386701226234436, 0.25078946352005005]

```

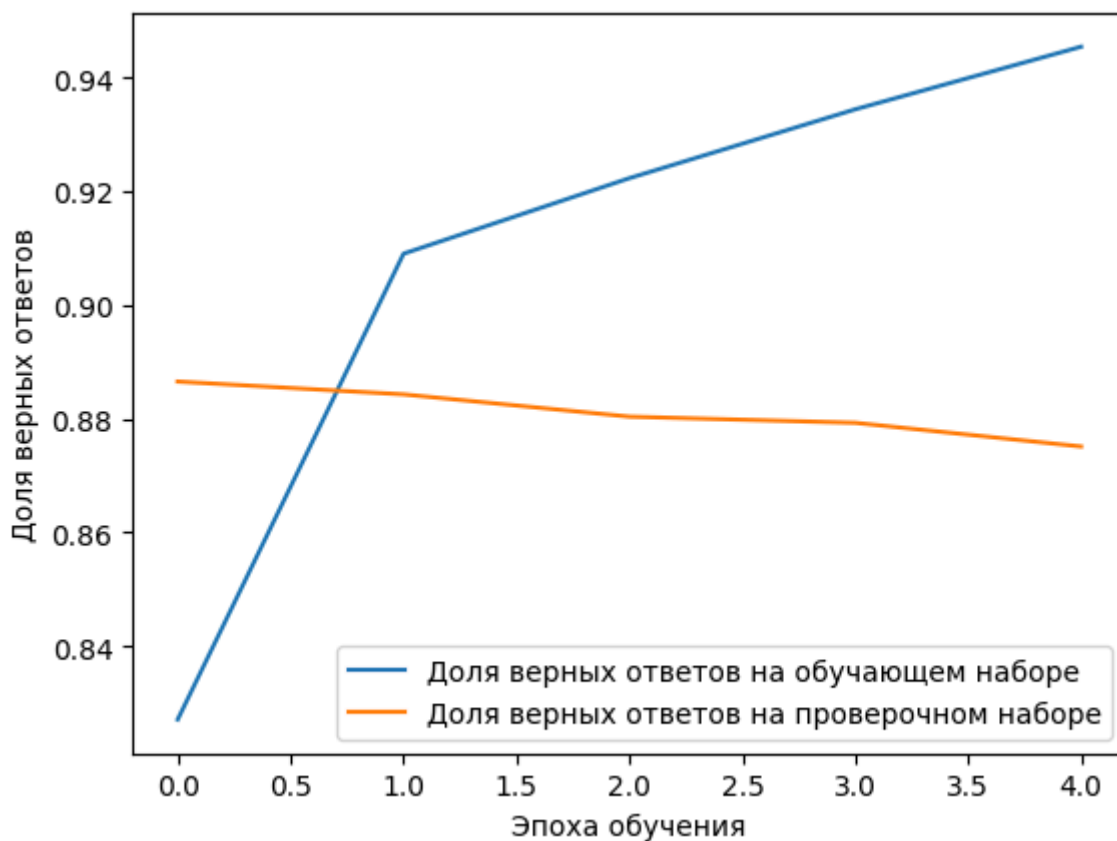


Рисунок 13 - Изменение ассигасу на датасете с базовой предобработкой.

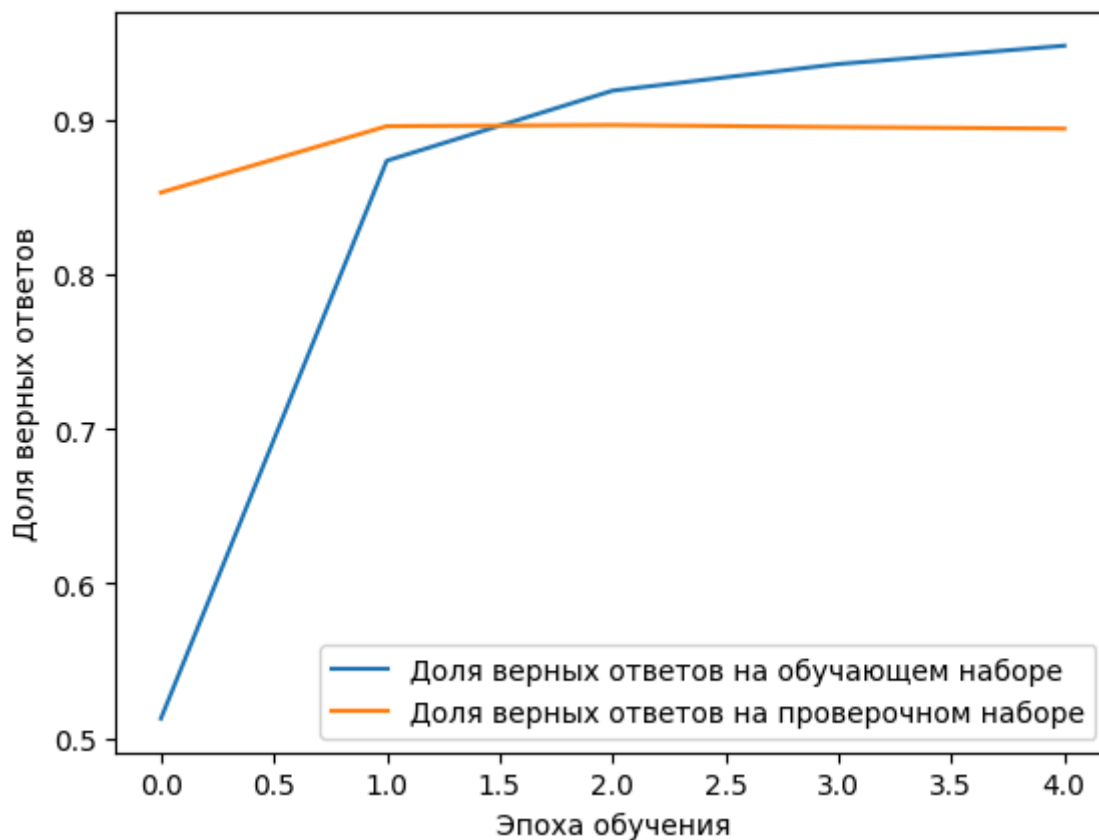


Рисунок 14 - Изменение ассигасу на датасете с добавлением заголовков.

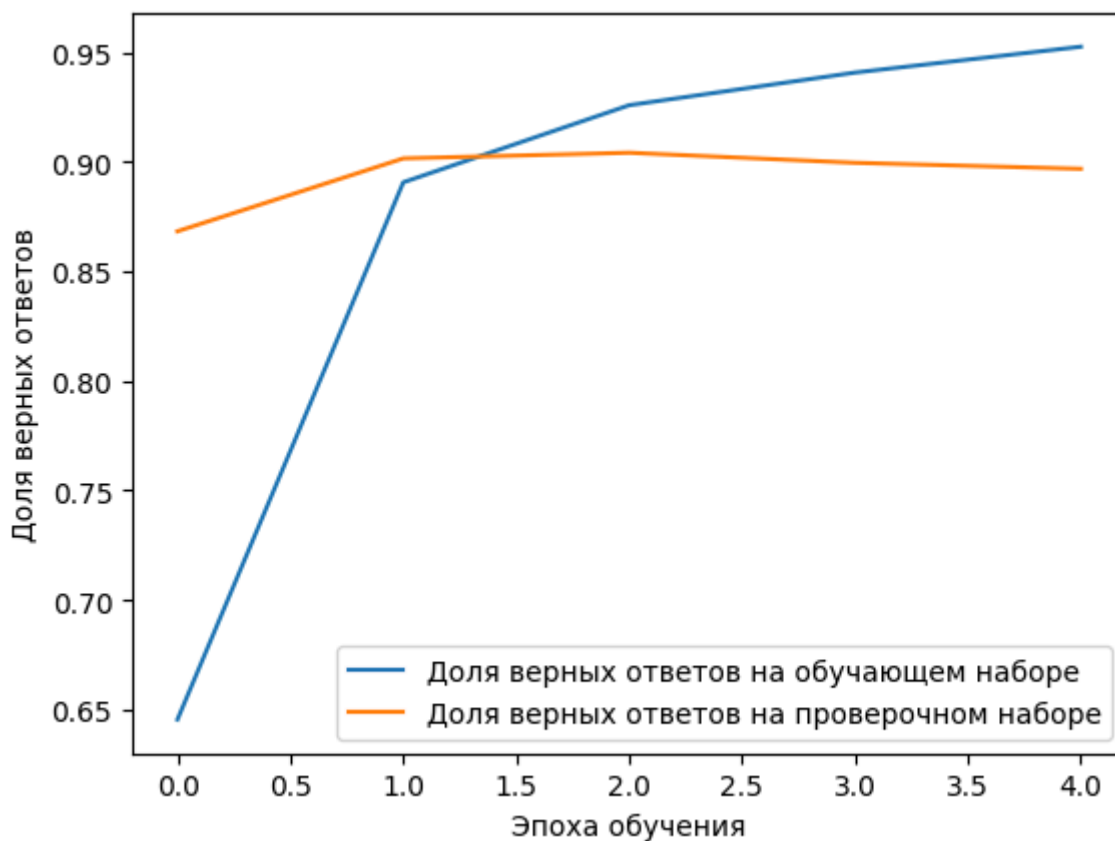


Рисунок 15 - Изменение ассигасу на датасете с удалением стоп-слов.

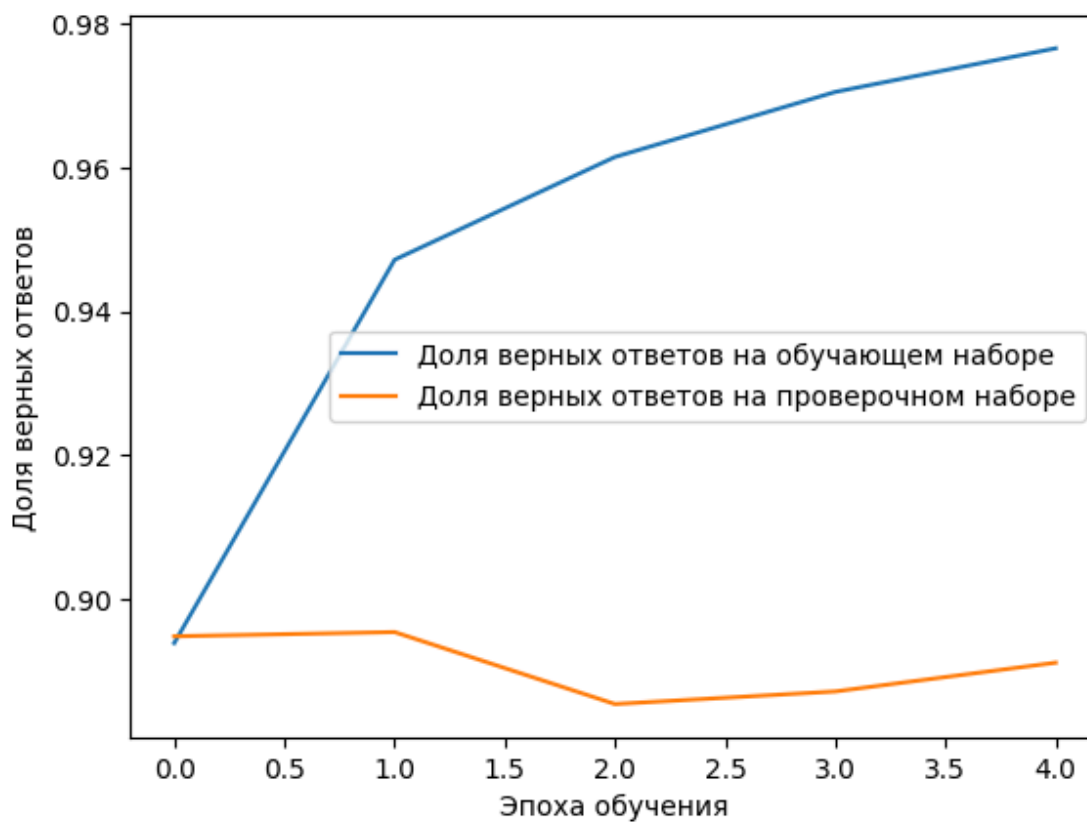


Рисунок 16 - Изменение ассигасу на датасете с лемматизацией.

Исходный код

Исходный полный код программы представлен в Google Colab по ссылке: [NeuroInf_lab3.ipynb](#)

Вывод

В ходе выполнения лабораторной работы было исследовано применение различных архитектур нейронных сетей для задачи классификации. При базовой предобработке текста все модели показали близкие результаты. Добавление заголовков новостей и увеличение длины входной последовательности в целом улучшили качество классификации, что подтверждает информативность заголовков. Очистка текста от стоп-слов и пунктуации привела к снижению точности всех моделей, что указывает на потерю полезного контекста. В то же время применение лемматизации положительно сказалось на результатах, повысив обобщающую способность моделей. Использование регуляризации в виде Dropout для свёрточной нейронной сети позволило уменьшить переобучение и сделать обучение более стабильным. В результате было показано, что наилучшие результаты достигаются при сочетании свёрточной архитектуры, использования заголовков, лемматизации текста и методов регуляризации.